

Outscaling Flyte v1

Flyte v2's Per-Action Architecture Runs Common Patterns up to 6.5× Faster

TECHNICAL REPORT, ORCHESTRATOR SCALABILITY STUDY, JUNE 2026

ABSTRACT

Flyte is a Kubernetes-native workflow orchestrator. Its original workflow engine (v1, flytepropeller, a data-plane controller) represents each workflow execution as a *single* custom resource (a `FlyteWorkflow` CRD) that one controller process reconciles in its entirety on every loop. This design is simple but couples a run's cost to the size of one object and concentrates all of a run's state in one shared, memory-bounded binary. The redesigned architecture (v2, a control-plane runs-service plus a data-plane executor) instead decomposes a run into many small *actions*, each its own compact record reconciled independently, so no single in-memory object holds the whole run. We benchmark the two designs on identical hardware (an 8 GiB engine pod on AWS EKS) across three workload shapes (wide fan-out, sustained concurrency, and long sequential chains) at scales from hundreds to forty thousand tasks. v2 completes the common patterns **4.3–6.5× faster** and, critically, exhibits *no* single-process memory cliff: its engine footprint stays flat at ~0.3 GiB (~4% of the limit) regardless of a run's width, whereas v1, holding a whole run in one CRD, drives the shared flyte-binary pod into the 8 GiB ceiling and OOM-kills at a ~6,000-leaf fan-out, an outage that affects every tenant. Spread instead across many concurrent workflows, both planes complete every run and scale sub-linearly once v1 is run at the same parallelism as v2 (`--max-parallelism 1000`): 40,000 held tasks (40 concurrent 1,000-task workflows) finish in ~17 minutes on v1 and ~13 minutes on v2 (v2 **~1.2–1.7×** faster across the range). Multi-workflow concurrency is the shape where the two planes are closest; v2's decisive advantages are the eliminated single-process cliff and single-workflow latency. We attribute v2's gains to (i) eliminating whole-object re-reconciliation, (ii) replacing per-workflow with per-action parallelism, and (iii) spreading a run's state across many small records instead of one in-memory object.

1 Introduction

Workflow orchestrators must turn a user's DAG into thousands of scheduled, monitored, retried tasks while a central orchestrator tracks state. As pipelines widen (large fan-outs) and platforms become multi-tenant, the orchestration engine, not the worker pods, becomes the scaling bottleneck and the blast radius for failures. An engine that concentrates a run's state in one process can be taken down by a single large job, degrading service for *all* users on a shared cluster.

This report quantifies exactly that effect for Flyte and shows how the v2 redesign removes it. We make three contributions: (1) a controlled, apples-to-apples benchmark of the v1 and v2 orchestration engines on the same cluster and the same workloads; (2) a measurement of v1's *OOM cliff*, the load at which the shared binary is killed, and a demonstration that v2 has no equivalent; and (3) an analysis tying the observed speedups and the cliff to one architectural decision: one

fat CRD per run versus many small, independently-reconciled action records.

2 Background: the v1 single-CRD engine

In v1, submitting a workflow creates one `FlyteWorkflow` custom resource that encodes the entire DAG and its live node state. A single controller, *flytepropeller*, owns the execution: on each reconcile loop it deep-copies and re-evaluates the whole object, then writes it back. *flyteadmin*, the scheduler, and *datacatalog* are co-located with *propeller* in one `flyte-binary` pod sharing an 8 GiB limit (Fig. 1, left). Two consequences follow directly:

Cost scales with object size. As a run fans out or lengthens, its one CRD grows; every reconcile re-processes the larger object, so round latency climbs with the live node set. We measured *propeller* round-time p99 of ~46 s under a 1,000-leaf fan-out.

Parallelism is per-workflow. One worker owns one workflow, so a single wide run cannot use the worker pool: 19 of 20 workers sat idle during a large fan-out. And because all of that run's state lives in one process,

its memory is bounded by the shared 8 GiB; cross that and the *whole* flyte-binary pod dies, taking the co-located control plane (flyteadmin) down with it.

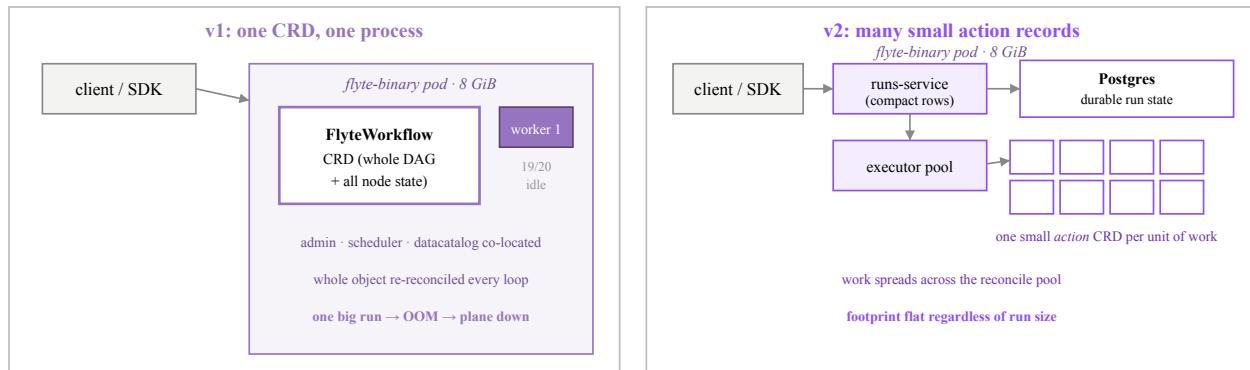


Figure 1. The two orchestration designs. Both run as a single `flyte-binary` pod under an identical 8 GiB limit. The difference is internal: **v1** concentrates an entire run in one `FlyteWorkflow` CRD reconciled by one worker inside a single shared 8 GiB binary. **v2** decomposes a run into many small *action* records reconciled by a pool, with per-action records, state and work both decomposed.

3 v2 design

v2 splits orchestration into a stateless, control-plane *runs-service* (the API and bookkeeping, writing compact rows to Postgres) and a data-plane *executor* that reconciles work (Fig. 1, right). The key change is granularity: each unit of work becomes an *action* with its own small CRD and its own reconcile, drawn from a shared work-queue pool. The run's live state is many small per-action records rather than one growing in-memory object. Three properties result, each the inverse of a v1 limitation: there is no single growing object to re-reconcile, so per-unit cost is flat; actions fan across the pool, so parallelism is real; and no run's state is pinned to one process, so there is no shared memory ceiling to hit.

4 Methodology

All runs execute on one shared AWS EKS cluster. The v1 and v2 engine pods each carry an 8 GiB (8192 MiB) memory limit. To isolate *orchestration* cost we use a `core-sleep` leaf task that runs in the engine with no task pod, so we measure the reconciler, not pod startup. Both systems run at `--max-parallelism 1000`.

Each cell runs once, sequentially; we wipe all CRDs and let pod memory settle between runs so no measurement is confounded by leftover state. We report end-to-end wall-clock (client-side submit→terminal), peak engine working-set memory (sampled live from the pod), and OOM events (container restart with exit 137). Table 1 lists the three workload shapes and their sweeps; Fig. 2 sketches each shape.

Table 1. Workload shapes, swept scales, and end-to-end runtime across the range (smallest→largest scale; both planes at `max-parallelism 1000`). *Leaf sleep* is each `core-sleep` task's hold time: 0 s for the timing-focused fan-out and long-chain shapes, and 120 s for concurrency, where the point is how many tasks the plane holds live at once.

Workload	Stresses	Scale swept	Leaf sleep	Flyte v1	Flyte v2
fanout	wide parallel breadth	1,000 → 6,000 leaves	0 s	124 → 717 s	29 → 115 s
concurrency	steady-state held load	1,000 → 40,000	120 s	242 → 1,016 s	146 → 756 s
long_chain	sequential length	100 → 500	0 s	74 → 365 s	16 → 56 s

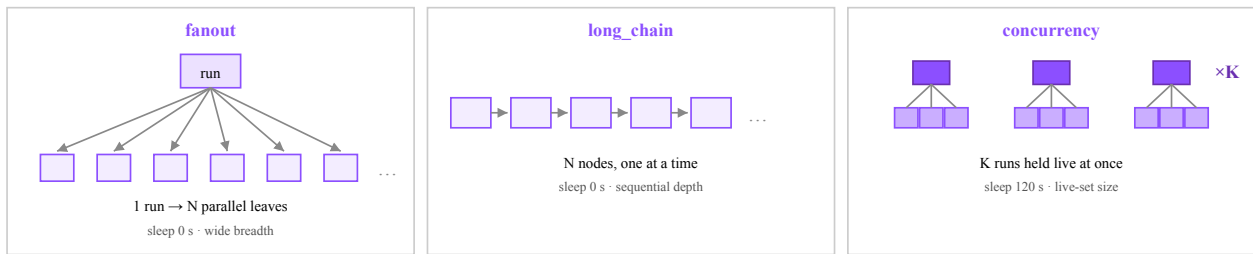


Figure 2. The three workload shapes. **fanout** stresses parallel breadth (one run, N simultaneous leaves); **long_chain** stresses sequential depth (N nodes advanced one at a time); **concurrency** stresses steady-state held load (K runs kept live at once). Fan-out and long-chain leaves sleep 0 s to isolate orchestration cost; concurrency leaves sleep 120 s so the plane must hold the full live set.

5 Evaluation

5.1 End-to-end latency

On the common patterns v2 is decisively faster. A 1,000-leaf fan-out drops from 124 s to 29 s (**4.3×**), widening to **6.4×** at 6,000 leaves (717 s → 115 s, Fig. 4); a 500-node sequential pipeline from 365 s to

56 s (**6.5×**, Fig. 3 right). The cause is structural: v1 reconciles one growing object on every loop, so a long chain pays repeatedly to re-evaluate an ever-larger CRD, while v2 advances one compact action at a time (Fig. 5). Sustained concurrency, the multi-workflow shape, is where the two planes are closest, though v2 stays modestly ahead (§5.4).

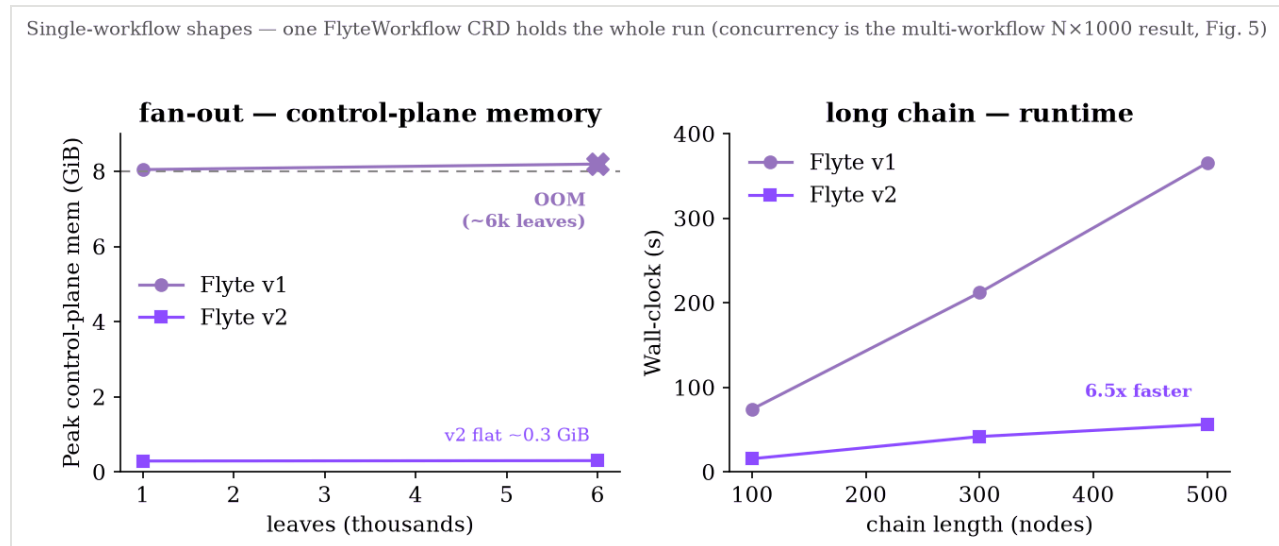


Figure 3. Per-experiment scaling, v1 vs v2 (single-workflow shapes, parallelism 1000). *fan-out*: engine memory vs leaves: v1 grazes the 8 GiB ceiling and OOM-kills at ~6,000 leaves while v2 stays flat at ~0.3 GiB. *long chain*: wall-clock vs length: v2 is 6.5× faster. Concurrency is the multi-workflow N×1000 result (Fig. 6).

5.2 Memory scaling and the OOM cliff

The reliability difference is starker than the latency one (Fig. 3, left). Because v1 holds every leaf of a run in one `FlyteWorkflow` CRD reconciled in the shared binary, engine memory tracks the run's width: a 1,000-leaf fan-out already pins the binary near 8 GiB, and by ~6,000 leaves it reaches the ceiling and is **OOM-killed** (exit 137). Because flyteadmin, the scheduler, and data-catalog share that binary, the API goes down for *every* tenant, not just the offending run. v2 decouples footprint from the run: it holds the same fan-outs flat at ~0.3 GiB (~4% of the limit) and never approaches a

ceiling. There is simply no single-process cliff to hit; v2's bound is throughput, not memory.

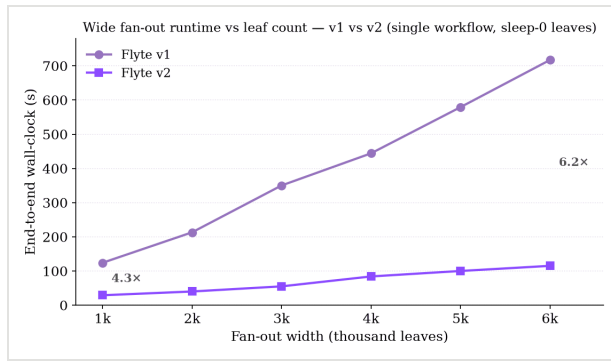


Figure 4. Wide fan-out end-to-end runtime vs leaf count (single workflow, sleep-0 leaves; wall-clock = submit → all leaves terminal). v2 stays **4.3–6.4×** faster across the range: 29 s vs 124 s at 1,000 leaves, 115 s vs 717 s at 6,000. v1's single reconcile loop churns the whole CRD each tick, so its runtime climbs steeply; under *held* leaves the same width hits the 8 GiB memory cliff and OOM-kills (Fig. 3A, §5.2).

5.3 Footprint under length

Even where v1 does *not* OOM, its footprint tracks the run while v2's stays flat. Across chain length 100→500, v1 memory grows 1272→1589 MiB while v2 holds near ≈320 MiB throughout (Fig. 5). Neither system OOMs on this shape within the swept range, but the trend shows why v1's headroom is workload-dependent and v2's is not.

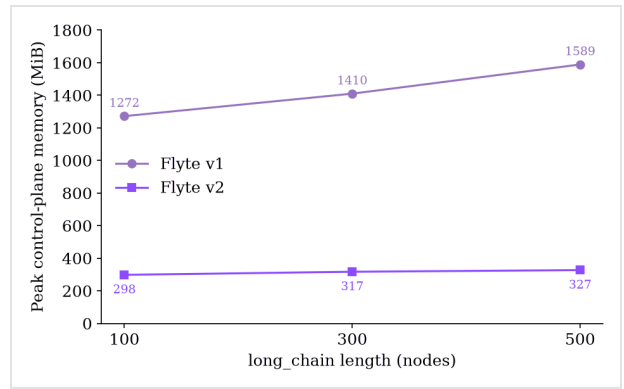


Figure 5. Footprint under length: v1's engine memory grows with the chain (1272→1589 MiB across length 100→500) because it re-reconciles one growing CRD, while v2 stays flat at ~320 MiB.

5.4 Concurrency across many workflows

The memory cliff (§5.2) is v1's ceiling when *one* workflow holds many leaves. A second, independent ceiling appears when load is spread across *many concurrent* workflows. We submit K workflows, each a 1,000-leaf fan-out held 120 s, and measure wall-clock from submit to all K terminal (Fig. 6). Here v1 no longer OOMs: each run's 1,000 leaves live in its own CRD, under the ~6,000-leaf single-CRD memory limit (§5.2). Run at the same `--max-parallelism 1000` as v2 (propeller's default caps in-flight nodes far lower, which would throttle these fan-outs), v1 keeps pace: 40 concurrent 1,000-task workflows (40,000 held tasks) complete in **~17 minutes** on v1 and **~13 minutes** on v2. Both planes scale sub-linearly and complete every run; v2 stays modestly ahead across the range (242→1,016 s on v1 vs 146→756 s on v2, a **~1.2–1.7×** edge). This is the workload shape where the two designs are closest: per-action decomposition does not multiply throughput here the way it removes the memory cliff (§5.2) or cuts single-workflow latency (§5.1).

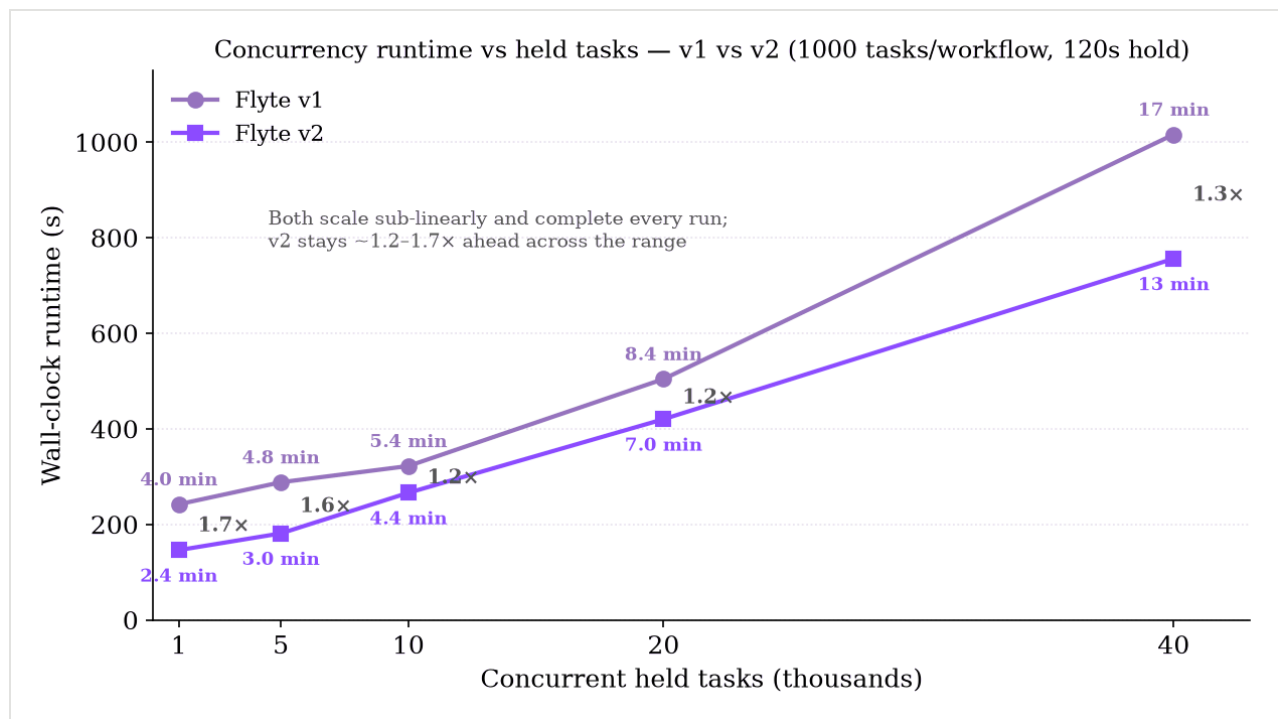


Figure 6. Concurrency runtime vs held tasks, the multi-workflow shape (K workflows $\times$ 1000 tasks each, leaves held 120 s; wall-clock = submit $\rightarrow$ all workflows terminal, single clean pass). Run at matched `max-parallelism 1000`, both planes complete every workflow and scale sub-linearly; **v2** stays $\sim 1.2\text{--}1.7\times$ ahead of **v1** across the range (146 $\rightarrow$ 756 s vs 242 $\rightarrow$ 1,016 s). This is the shape where the two designs are closest.

6 Discussion

The results reduce to a single principle: *where you put a run's state determines how it scales*. v1 keeps state in one in-memory object in one process, which makes orchestration cost grow with run size, caps parallelism at one worker per run, and, most consequentially on a shared cluster, turns one heavy user into a platform-wide outage. v2 keeps state in many small records, trading a small constant per-action overhead for flat memory, real parallelism, and a blast radius contained to the run. For multi-tenant platforms the elimination of the OOM cliff is more valuable than the raw speedup: it converts "a big job can take everyone down" into "the platform absorbs it."

7 Conclusion

Re-architecting Flyte's orchestration from one fat CRD per run to many small, independently-reconciled actions makes the common workflow patterns 4.3–6.5 $\times$ faster and removes v1's single-process memory cliff: where v1 drives the shared flyte-binary pod into its 8 GiB ceiling and OOM-kills at a $\sim 6,000$ -leaf fan-out, v2 holds any run flat at ~ 0.3 GiB with zero engine crashes. On multi-workflow concurrency the two planes are closest: run at matched parallelism, both complete every run to 40,000 held tasks and scale sub-linearly, with v2 a modest $\sim 1.2\text{--}1.7\times$ ahead.